Integração de Serviços

A nossa API como cliente de outra API: a previsão do tempo no cadastro de tarefas

Conceito → cliente HTTP, contrato dos outros, 502 e 504 • Prática → RestClient + Open-Meteo

AGENDA

O que vamos ver hoje

- **Parte 1, a nossa API como cliente:** o que é integração de serviços e o contrato dos outros.
- **Parte 2, o Open-Meteo e a rota /previsao:** a API de previsão do tempo, as regras e a localização fixa (Criciúma).
- **Parte 3, RestClient na prática:** montar o cliente, fazer a chamada, traduzir a resposta e configurar timeouts.
- **Parte 4, quando o outro lado falha:** 502, 504, regras antes da chamada e dado de fora.
- Vamos sair hoje com o cadastro de tarefas mostrando a previsão do tempo para o dia do prazo.

1. A nossa API como cliente

Servidor e cliente ao mesmo tempo

2. O Open-Meteo e a rota /previsao

Uma lista por medida, regras R1 a R3, localização fixa

3. RestClient na prática

get(), uri(), retrieve(), body() e timeouts

4. Quando o outro lado falha

502, 504 e o dado que não veio

RECAPITULANDO

De onde viemos

- Na Aula 08, as tarefas foram para o PostgreSQL. Na Aula 09, ganharam DTOs, mapper e Swagger.
- Em todas as rotas, a API fazia a mesma coisa: recebia o pedido, consultava o **próprio banco** e respondia.
- Ela nunca perguntava nada para outro sistema.
- Hoje: no meio de uma requisição, a API faz **outra requisição HTTP**, para outra API, e usa a resposta.

service/TarefaService.java (Aula 09)

```
// Aula 09 -- TarefaService
public TarefaResponseDTO criar(TarefaRequestDTO dto) {
    Responsavel responsavel =
            buscarResponsavel(dto.responsavelId());
    Tarefa tarefa = mapper.toEntity(dto, responsavel);
    Tarefa salva = repository.save(tarefa);
    return mapper.toResponse(salva);
}

// Tudo o que o Service usa mora aqui dentro:
// Repository, Mapper e o nosso PostgreSQL.
```

PARTE 1

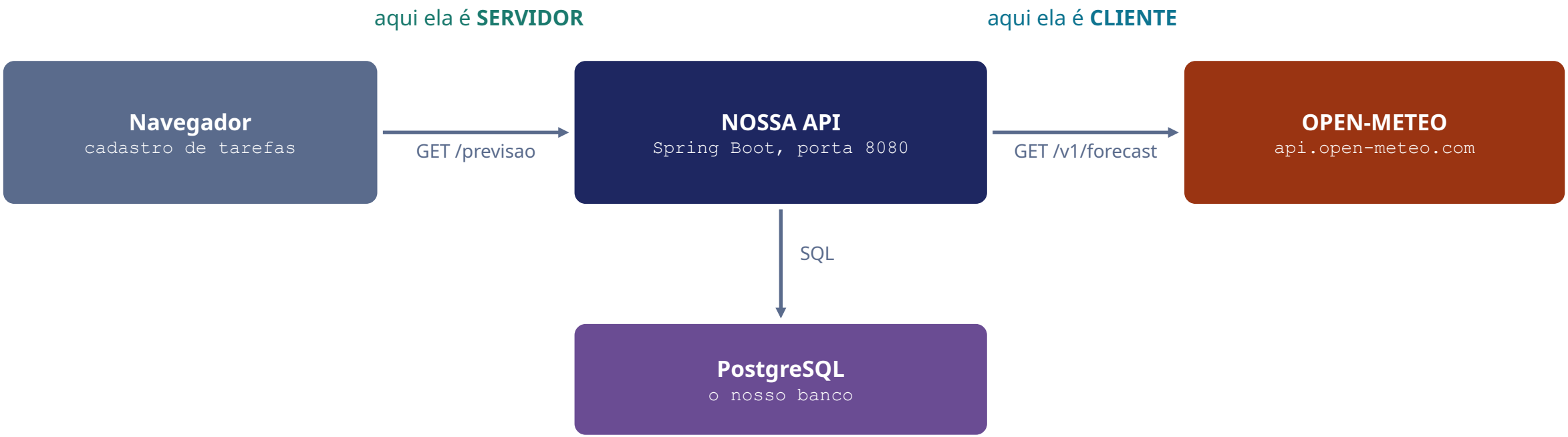
A nossa API como cliente

No meio de uma requisição, ela faz outra requisição

CONCEITO

O que é integração de serviços

Um sistema usando outro sistema pela rede, através da API dele. A mesma API passa a ter **dois papéis**: é servidor para quem a chama e cliente do serviço que ela chama.



Tudo o que vimos de HTTP (verbos, status, JSON, contrato) vale dos dois lados da nossa API.

NO DIA A DIA

Integração está em todo lugar



Frete

A loja virtual pergunta aos Correios quanto custa e quanto demora a entrega.



CEP

O cadastro manda o CEP e recebe rua, bairro e cidade de uma API de endereços.



Pagamento

A compra só fecha depois que a operadora do cartão responde que aprovou.



Login com Google

O site pergunta ao Google se aquela pessoa é mesmo quem diz ser.



Nesta aula: ao escolher a data do prazo de uma tarefa, a nossa API pergunta ao **Open-Meteo** como vai estar o tempo naquele dia, no lugar onde a pessoa está.

CONCEITO

O contrato dos outros

- Quando a API é nossa, **o contrato é nosso**: escolhemos nomes, campos e status (Aula 09).
- Quando a API é dos outros, **o contrato é dado**: a gente lê, se adapta e não tem como mudar.
- O Open-Meteo fala inglês com _, manda **uma lista por medida** e diz o tempo com um número (95 é trovoada).
- A nossa API continua falando português. Quem traduz é o **PrevisaoMapper**.

o Open-Meteo responde (o contrato DELE)

```
"daily": {
  "time": ["2026-10-24"],
  "weather_code": [95],
  "temperature_2m_max": [28.0],
  "temperature_2m_min": [17.0],
  "precipitation_probability_max": [75]
}
```

GET /previsao (o NOSSO contrato)

```
{
  "data": "2026-10-24",
  "descricao": "Trovoada",
  "icone": "⛈️",
  "temperaturaMaxima": 28.0,
  "chanceDeChuva": 75, ...
}
```

REFERÊNCIA

Onde cada coisa mora

Pacote	Classe	O que faz	Conhece
client/	`OpenMeteoClient`	O único lugar que sabe o endereço do Open-Meteo, o caminho, os parâmetros e que a chamada usa o RestClient	O Open-Meteo
client/	`OpenMeteoRespostaDTO`	O contrato do Open-Meteo, copiado do jeito que ele é	Nada
mapper/	`PrevisaoMapper`	Traduz: o item i de cada lista vira um dia; weather_code vira descricao e icone	Os dois formatos
service/	`PrevisaoService`	As regras R1 a R3, antes de chamar o Open-Meteo	Client e Mapper
dto/	`PrevisaoDTO`	O NOSSO contrato para a previsão de um dia, em português	Nada
controller/	`PrevisaoController`	A rota GET /previsao. Não sabe que existe um Open-Meteo por trás	Service e DTOs



Se a previsão passar a vir de outro serviço, **só o client/ e o mapper mudam**. É o "pacote novo para o cliente da API externa" previsto na arquitetura do Projeto Final (ARQUITETURA.md, seção 9).

CONCEITO

O caminho de um GET /previsao



Ida: 1. o Controller converte a data e repassa → 2. o Service confere as regras → 3. só então o client chama o Open-Meteo



Volta: 4. o Service confere se o dia veio → 5. o Mapper traduz as listas no NOSSO formato → 6. o Jackson transforma o record em JSON

Uma chamada ao Open-Meteo

por GET /previsao

Regras antes da chamada

pedido inválido não gasta requisição

Tradução no mapper

o formato do outro não vaza para o nosso

PARTE 2

O Open-Meteo e a rota /previsao

A previsão do tempo no cadastro de tarefas

NA PRÁTICA

O Open-Meteo: previsão do tempo gratuita

- Um serviço de previsão do tempo **gratuito e sem cadastro**: não pede chave de acesso.
- Uma rota só, `/v1/forecast`, e tudo vai na query string: o lugar (`latitude`, `longitude`), o que se quer (`daily=...`) e o período (`start_date`, `end_date`).
- Prevê **16 dias**: hoje e mais 15.
- Dá para abrir no navegador: é só um GET.

navegador: api.open-meteo.com

```
# GET /v1/forecast?latitude=-28.6775
#   &longitude=-49.3697&timezone=auto
#   &daily=weather_code,temperature_2m_max
#   &start_date=2026-10-24&end_date=2026-10-24
{
  "latitude": -28.64675,
  "longitude": -49.36145,
  "timezone": "America/Sao_Paulo",
  "generationtime_ms": 0.17,
  "elevation": 49.0,
  "daily_units": { ... },
  "daily": {
    "time": ["2026-10-24"],
    "weather_code": [95],
    "temperature_2m_max": [28.0]
  }
}
```

CONCEITO

Uma lista por medida

O `daily` não é uma lista de dias: é um objeto com **uma lista por medida**, todas do mesmo tamanho. O dia `i` é o item `i` de cada lista.

lista do daily	[0]	[1]	[2]
`time`	**"2026-10-24"**	"2026-10-25"	"2026-10-26"
`weather_code`	**95**	80	3
`temperature_2m_max`	**28.0**	18.3	22.1
`temperature_2m_min`	**17.0**	14.1	13.5
`precipitation_probability_max`	**75**	86	37



```
PrevisaoMapper.toPrevisao(resposta, 0)

{
  "data": "2026-10-24",
  "descricao": "Trovoada",
  "icone": "⛈️",
  "temperaturaMaxima": 28.0,
  "temperaturaMinima": 17.0,
  "chanceDeChuva": 75
}
```

É por isso que o mapper recebe o índice do dia: no GET /previsao é sempre 0 (um dia só); no Exercício 5, são 16 dias de uma vez.

NA PRÁTICA

Do código WMO ao texto

weather_code	descricao	icone
0	Céu limpo	☀️
1, 2, 3	Poucas nuvens, parcialmente nublado, nublado	🌤️ ⛅ ☁️
51, 53, 55	Garoa	🌦️
61, 63, 65	Chuva fraca, moderada, forte	🌧️
80, 81, 82	Pancadas de chuva	🌦️
95, 96, 99	Trovoada (com granizo)	⛈️

mapper/PrevisaoMapper.java

```
// "switch" com setas (Java 14+): cada case
// devolve um valor, sem break.
return switch (codigo) {
    case 0 -> "Céu limpo";
    case 1 -> "Poucas nuvens";
    case 61 -> "Chuva fraca";
    case 95 -> "Trovoada";
    // ... os outros códigos ...
    // um código que a tabela não conhece:
    // mostra o número, não quebra
    default -> "Código do tempo " + codigo;
};
```

A tabela é a da Organização Meteorológica Mundial (WMO), que o Open-Meteo usa. Traduzir aqui poupa todo cliente da nossa API de conhecer esses números.

NA PRÁTICA

A rota GET /previsao

```
GET /previsao?latitude=-28.6775&longitude=-49.3697&data=2026-10-24

# 200 OK
{
  "data": "2026-10-24",
  "descricao": "Trovoada",
  "icone": "⛈️",
  "temperaturaMinima": 17.0,
  "temperaturaMaxima": 28.0,
  "chanceDeChuva": 75,
  "ventoMaximo": 15.9,
  "latitude": -28.64675,
  "longitude": -49.36145
}
```

Três parâmetros na URL

latitude e longitude são fixas (Criciúma, no código da página); data vem do campo do prazo, no formato yyyy-MM-dd.

@DateTimeFormat(iso = DATE)

Sem essa anotação no @RequestParam, o Spring tentaria o formato do idioma do servidor e recusaria "2026-10-24".

A latitude que volta é outra

O Open-Meteo arredonda para a grade dele: -28.6775 vira -28.64675.

As regras vêm antes da chamada

R1. Coordenadas do planeta

Latitude entre -90 e 90; longitude entre -180 e 180.

R2. De hoje em diante

Previsão é sobre o futuro: data de ontem não tem previsão.

R3. Até 15 dias depois de hoje

O Open-Meteo prevê 16 dias, contando hoje.

service/PrevisaoService.java

```
validarCoordenadas(latitude, longitude);    // R1
validarData(data);                          // R2 e R3
OpenMeteoRespostaDTO resposta =
        client.buscarPrevisao(latitude, longitude, data, data);
```

Pedido inválido não sai daqui

400 com a mensagem da NOSSA regra, sem gastar uma requisição pela internet.

NA PRÁTICA

A localização é fixa: Criciúma/SC

- Antes, quem sabia onde a pessoa está era o **navegador** (GPS, Wi-Fi ou rede), com a permissão dela.
- No laboratório isso **não funciona**: exige permissão e contexto seguro (http://localhost ou HTTPS).
- Por isso a página usa uma localização **fixa**: a SATC, em Criciúma.
- A página manda para a nossa API **só dois números**. O backend continua sem saber "onde o usuário está".

static/index.html

```
// localização fixa: Criciúma/SC (SATC)
const LOCALIZACAO = {
  nome: 'Criciúma/SC',
  latitude: -28.6775,
  longitude: -49.3697,
};

fetch(`/previsao`
  + `?latitude=${LOCALIZACAO.latitude}`
  + `&longitude=${LOCALIZACAO.longitude}`
  + `&data=${data}`);
```

Para usar outra cidade, é só **trocar a latitude e a longitude** da constante. O resto do fluxo (a página, a nossa API e o Open-Meteo) não muda.

A previsão no cadastro de tarefas



- Escolheu a data do prazo: a previsão daquele dia aparece logo abaixo do campo.
- O log mostra o caminho todo, com as camadas **CLIENT** (a nossa classe) e **EXTERNA** (o Open-Meteo).
- A barra "Aula 12" marca cada exercício que já está no ar.

PARTE 3

RestClient na prática

O cliente HTTP do Spring, sem nenhuma dependência nova

REFERÊNCIA

Qual cliente HTTP usar

Cliente	O que é	Na disciplina
`RestClient`	O cliente HTTP do Spring 6.1+ (Spring Boot 3.2+). Escrita encadeada, síncrono	É o que usamos. Já vem no spring-boot-starter-web
`RestTemplate`	O cliente antigo do Spring. Aparece muito em código existente	Em manutenção: para código novo, o Spring recomenda o RestClient
`WebClient`	Cliente reativo (assíncrono), do Spring WebFlux	Outra dependência e outro modelo de programação
`@HttpExchange`	Interface HTTP: você escreve só a interface, e o Spring gera a implementação	Parecido com o Spring Data JPA, só que para HTTP. Vale conhecer



Nenhuma linha nova no pom.xml nesta aula. O RestClient faz parte do spring-web, que o spring-boot-starter-web já traz desde a Aula 07.

NA PRÁTICA

Montando o OpenMeteoClient

- `@Component`: o Spring cria um OpenMeteoClient e injeta no PrevisaoService.
- `RestClient.Builder`: um "montador" que o Spring Boot já entrega com os timeouts e o mesmo Jackson da nossa API.
- `@Value("${previsao.url}")`: o valor da linha previsao.url do application.properties.
- `baseUrl`: cada chamada só diz o caminho (/v1/forecast); o começo do endereço vem daqui.

client/OpenMeteoClient.java

```
@Component
public class OpenMeteoClient {

    private final String url;
    private final RestClient restClient;

    public OpenMeteoClient(RestClient.Builder builder,
            @Value("${previsao.url}") String url) {
        this.url = url;
        this.restClient = builder
                .baseUrl(url)
                .build();
    }
}
```

NA PRÁTICA

Anatomia de uma chamada

client/OpenMeteoClient.java

```
return chamar("buscar a previsão do tempo",
    () -> restClient.get()
        .uri(uriBuilder -> uriBuilder
            .path("/v1/forecast")
            .queryParam("latitude", latitude)
            .queryParam("longitude", longitude)
            .queryParam("daily", MEDIDAS_DIARIAS)
            .queryParam("timezone", "auto")
            .queryParam("start_date", inicio)
            .queryParam("end_date", fim)
            .build())
        .retrieve()
        .body(OpenMeteoRespostaDTO.class));
```

`get()`

O verbo HTTP (há também post(), put(), patch(), delete()).

`uri(uriBuilder -> ...)`

O caminho e cada queryParam, já codificados. O builder começa do baseUrl.

`retrieve()`

Envia e recebe. Status 4xx e 5xx viram exceção.

`body(...)`

Converte o JSON da resposta no record pedido.

NA PRÁTICA

Ler só o que precisa

o Open-Meteo responde

```
{
  "latitude": -28.64675,
  "longitude": -49.36145,
  "generationtime_ms": 0.17,
  "utc_offset_seconds": -10800,
  "timezone": "America/Sao_Paulo",
  "timezone_abbreviation": "GMT-3",
  "elevation": 49.0,
  "daily_units": { ... },
  "daily": { ... }
}
```



client/OpenMeteoRespostaDTO.java

```
// Só o que a nossa API usa.
public record OpenMeteoRespostaDTO(
        Double latitude,
        Double longitude,
        String timezone,
        Diario daily
) { ... }

// O resto é ignorado pelo Jackson:
// no Spring Boot, campo desconhecido
// no JSON não é erro.
```

Se o Open-Meteo acrescentar um campo amanhã, nada quebra aqui. Ler só o que precisa deixa a integração menos frágil.

NA PRÁTICA

O PrevisaoMapper: um dia de cada lista

mapper/PrevisaoMapper.java

```
public PrevisaoDTO toPrevisao(OpenMeteoRespostaDTO resposta,
                              int indice) {
    OpenMeteoRespostaDTO.Diario dia = resposta.daily();
    Integer codigo = dia.weather_code().get(indice);
    return new PrevisaoDTO(
            dia.time().get(indice),
            descricao(codigo),
            icone(codigo),
            dia.temperature_2m_min().get(indice),
            dia.temperature_2m_max().get(indice),
            dia.precipitation_probability_max().get(indice),
            dia.wind_speed_10m_max().get(indice),
            resposta.latitude(),
            resposta.longitude());
}
```

- O único lugar que conhece **os dois vocabulários**: `temperature_2m_max` de um lado, `temperaturaMaxima` do outro.
- Igual aos mappers da Aula 09: **só converte**. Não chama o Open-Meteo nem o banco.
- Os nomes com `_` no record do Open-Meteo são de propósito: são os nomes do JSON dele.

Timeouts: quanto tempo esperar

2 s

connect-timeout

para conseguir conectar no Open-Meteo

5 s

read-timeout

para a resposta chegar, depois de conectado

src/main/resources/application.properties

```
spring.http.client.connect-timeout=2s
spring.http.client.read-timeout=5s
```

- Sem essas linhas, uma chamada a um serviço travado espera por minutos, ou para sempre.
- E a requisição de quem chamou a NOSSA API fica presa junto, ocupando o Tomcat.
- O `RestClient.Builder` do Spring Boot já vem com esses valores: o OpenMeteoClient não faz nada a mais.
- Estourou o tempo: `HttpTimeoutException` (Exercício 3).

PARTE 4

Quando o outro lado falha

Fora do ar, lento, ou respondendo o que não esperávamos

REFERÊNCIA

Cada falha, um status

O que aconteceu no Open-Meteo	O que o RestClient lança	O que a nossa API responde
Respondeu 400 ou 500	`HttpClientErrorException` / `HttpServerErrorException`	**502 Bad Gateway**
Está fora do ar, ou o endereço está errado	`ResourceAccessException` (causa: ConnectException)	**502 Bad Gateway**
Demorou mais que o read-timeout	`ResourceAccessException` (causa: HttpTimeoutException)	502; no Exercício 3, **504 Gateway Timeout**
Respondeu um JSON que não bate com o record	`RestClientException`	**502 Bad Gateway**



Um catch da mãe pega todas as filhas: é o que o chamar() do OpenMeteoClient faz.

CONCEITO

Por que 502, e não 400 ou 500?

400



Bad Request

O cliente errou. Não foi isso: quem chamou a nossa API mandou tudo certo.

500



Internal Server Error

Bug nosso. Também não: o nosso código funcionou direitinho.

502



Bad Gateway

Sou um intermediário, e quem estava atrás de mim falhou. Foi exatamente isso.



E o **504 Gateway Timeout** é o primo do 502: o serviço de trás não respondeu a tempo (Exercício 3).

NA PRÁTICA

Traduzir as exceções num lugar só

client/OpenMeteoClient.java

```
private <T> T chamar(String operacao,
                     Supplier<T> chamada) {
    try {
        return chamada.get();
    } catch (RestClientException e) {
        // vira PrevisaoIndisponivelException → 502
        throw indisponivel(operacao, e);
    }
}
```

- Toda chamada passa pelo `chamar()`. A requisição vai como lambda, igual ao `orElseThrow(() -> ...)`.
- Qualquer exceção do RestClient vira **PrevisaoIndisponivelException**, e o ApiExceptionHandler responde 502.
- O Service e o handler nem sabem que existe RestClient.

A mensagem do ErroDTO

"Não foi possível buscar a previsão do tempo: sem resposta válida de https://api.open-meteo.com (ConnectException)". O que se tentou fazer, e a pista do motivo: ConnectException (fora do ar, endereço errado), HttpTimeoutException (demorou).

CUIDADO

Dado de fora não é confiável

- Mesmo respondendo 200, o outro sistema pode mandar menos do que esperamos.
- A nossa API se protege onde dá, e decide o que fazer em cada caso: **mostrar algo**, ou **falhar com clareza**.
- O que ela nunca faz: deixar estourar um NullPointerException e responder 500.



Código do tempo desconhecido

Vira "Código do tempo 42", com o ícone ❔. Não quebra.



Chance de chuva null

O campo vem null no PrevisaoDTO, e a tela simplesmente não mostra.



Resposta sem o dia pedido

O PrevisaoService lança PrevisaoIndisponivelException: 502.

ATENÇÃO

Timeout quer dizer "não sei"

Quando o outro serviço demora e a nossa API desiste de esperar, ela **não sabe** o que aconteceu do outro lado. O pedido pode ter:

1

Nem chegado lá

A rede falhou no caminho. Nada aconteceu.

2

Chegado e falhado lá

O serviço recebeu, mas não conseguiu responder.

3

Chegado e dado certo, só que tarde

O serviço terminou depois que a nossa API desistiu e respondeu 504.

Numa **consulta**, como a previsão, tanto faz: é só perguntar de novo. Num pedido que **grava** alguma coisa lá (um pagamento, um pedido de compra), o caso 3 faz toda a diferença: repetir pode fazer duas vezes. Por isso as operações **idempotentes** (Aula 07) valem tanto.

EXTRA

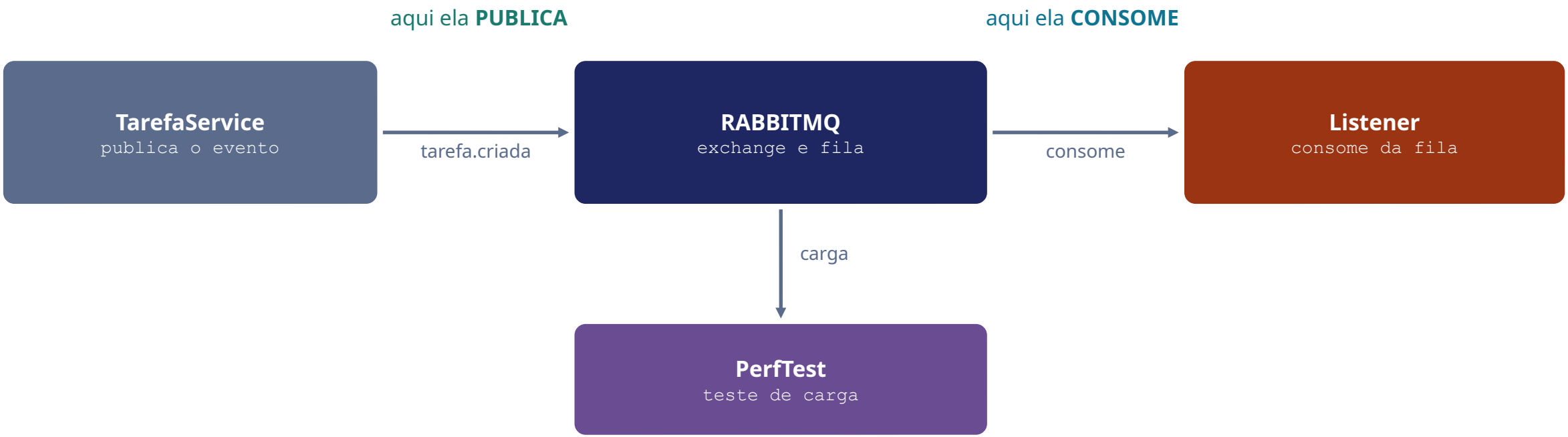
Mensageria com RabbitMQ

Avisar sem esperar: eventos entre as partes da API

CONCEITO

Mensageria: avisar sem esperar

Em vez de uma parte chamar a outra e ficar esperando, ela **publica um evento** num intermediário, o **broker**, e segue a vida. Quem se interessa **consome** quando puder.



A tarefa já está salva no banco antes do evento: se o RabbitMQ estiver fora do ar, só aparece um aviso no log e o cadastro não falha.

Publicar e consumir com Spring AMQP

- O **TarefaService** chama o **Publisher**, que manda o evento para a exchange **tarefas.eventos** com a routing key **tarefa.criada**, **tarefa.concluida**...
- A **exchange** (tipo topic) roteia pela routing key: a fila **tarefas.notificacoes** recebe tudo que casa com **tarefa.***.
- O **Listener** lê a fila sozinho, em outra thread: **não trava** a requisição HTTP.
- Se o broker cair, a tarefa **já foi salva**: o Publisher só registra um aviso no log.

TarefaEventoPublisher.java

```
try {
    rabbitTemplate.convertAndSend(
            EXCHANGE, "tarefa." + tipo, evento);
} catch (AmqpException e) {
    // broker fora do ar: só avisa no log
    log.warn("Não foi possível publicar...");
}
```

TarefaEventoListener.java

```
@RabbitListener(queues = "tarefas.notificacoes")
public void receber(TarefaEventoDTO evento) {
    log.info("[NOTIFICAÇÃO] Tarefa {}: {}",
            evento.tarefaId(), evento.tipo());
}
```

NA PRÁTICA

Teste de carga na fila

docker compose (Windows, Linux e Mac)

```
# PowerShell (Windows)
$env:CARGA_TAXA=500
$env:CARGA_PRODUTORES=2
docker compose --profile carga run --rm perftest

# bash (Linux / Mac)
CARGA_TAXA=500 CARGA_PRODUTORES=2 \
  docker compose --profile carga run --rm perftest

# resultado de um teste
sending rate avg: 1054 msg/s
tarefas.notificacoes  10009 mensagens
```

Variáveis (todas opcionais)

CARGA_TAXA (msg/s por produtor), CARGA_SEGUNDOS, CARGA_PRODUTORES e CARGA_CONSUMIDORES (0 = a API consome).

Sem script, sem passo extra

O RabbitMQ já cria a exchange e a fila ao subir (carga/definitions.json): roda até com a API desligada.

Veja a fila encher e esvaziar

Painel em localhost:15672 (aba Queues). Ligue a API e a fila esvazia.

NA PRÁTICA

Rodando o exemplo_tarefas da Aula 12

- Banco na porta **5439**, projeto Compose aula12-tarefas: separado das Aulas 08 e 09.
- Nenhuma migration nova: a previsão não é guardada no banco.
- Precisa de **internet**. A API sobe sem ela (a primeira chamada ao Open-Meteo só acontece quando alguém pede uma previsão), mas a previsão não.
- Abra a página em **localhost**: a previsão usa a localização fixa de Criciúma, sem pedir nada ao navegador.

terminal

```
cd "Aula 12/exemplo_tarefas"
docker compose up -d      # PostgreSQL, 5439
./mvnw spring-boot:run    # Windows: .\mvnw.cmd

# no navegador
http://localhost:8080/
http://localhost:8080/contrato.html
http://localhost:8080/swagger-ui.html

# testar direto: GET /previsao no Swagger UI,
# ou na collection do Postman
```

contrato.html: confira cada exercício



- A página roda 18 testes contra a SUA API, com as coordenadas da SATC (sem pedir a localização).
- Cria tarefas temporárias, testa e apaga no final.
- Projeto recém-baixado: 7 ✅ e 11 ⏳. Meta da entrega: tudo ✅.
- É a página que será aberta na correção.

EX 1 OUTRA CHAMADA

O tempo de agora

Objetivo: fazer outra chamada à mesma API, com outro formato de resposta.

- OpenMeteoAgoraDTO no formato do Open-Meteo: `current` é um objeto só, não listas.
- `OpenMeteoClient.buscarAgora()`: mesmo caminho, com `current=temperature_2m,wind_speed_10m,weather_code`.
- AgoraDTO, `PrevisaoMapper.toAgora()` e GET /previsao/agora (só a R1 vale: não há data).
- **1.1** Por que a nossa API não repassa as listas do daily do jeito que vieram?
- **1.2** Por que a página pede à NOSSA API, e não direto ao Open-Meteo?

Pronto quando...

- GET /previsao/agora: 200 com horario, descricao, icone, temperatura e vento
- latitude=999: 400
- A caixa da previsão mostra "Agora: ..."

EX 2 DADO PRÓPRIO + EXTERNO

Previsão para o prazo da tarefa

Objetivo: juntar a tarefa (nosso banco) com a previsão (Open-Meteo) numa resposta só.

- TarefaComPrevisaoDTO: tarefa, previsao e aviso (um DTO composto, como na Aula 09).
- `PrevisaoService.temPrevisao(data)` e `validarCoordenadas()` público.
- `TarefaService.buscarComPrevisao()`: sem previsão para o prazo, a tarefa vem com `previsao` null e um aviso.
- **2.1** Por que 30 dias dá 400 no GET /previsao, mas 200 com aviso aqui?
- **2.2** Por que usar o PrevisaoService, e não o OpenMeteoClient direto?

Pronto quando...

- Prazo em 3 dias: a tarefa e a previsão do dia do prazo
- Prazo em 30 dias: 200, previsao null e aviso
- Tarefa 999999: 404

EX 3 TIMEOUT

Tempo esgotado vira 504

Objetivo: separar "o serviço falhou" (502) de "o serviço demorou" (504).

- PrevisaoTempoEsgotadoException, **filha** da PrevisaoIndisponivelException.
- No `OpenMeteoClient.indisponivel()`: causa `HttpTimeoutException` vira a exceção nova.
- Tratador de 504 (GATEWAY_TIMEOUT) e @ApiResponse 504 nas rotas da previsão.
- Teste: `spring.http.client.read-timeout=1ms` (e volte para 5s depois).
- **3.1** Por que filha, e não RuntimeException? **3.2** E com read-timeout de 60 s e 200 pessoas?

Pronto quando...

- read-timeout=1ms: 504
- Endereço inexistente: 502, como antes
- Swagger mostra 200, 400, 502 e 504

EX 4 CACHE

Cache de 30 minutos

Objetivo: não perguntar de novo o que acabou de ser respondido.

- PrevisaoDTO ganha `origem` ("open-meteo" ou "cache") e o método `comOrigem()`.
- No PrevisaoService: `Map<String, Guardada>` (ConcurrentHashMap) com a previsão e o instante.
- Chave: latitude e longitude com 2 casas + a data. Menos de 30 minutos: responde do cache.
- **4.1** 30 alunos, 10 trocas de data entre 3 dias: quantas chamadas, com e sem cache? Qual o preço?
- **4.2** E se a chave usasse todas as casas? E 0 casas?

Pronto quando...

- 1ª vez: "origem": "open-meteo"
- Mesmo pedido de novo: "origem": "cache"
- A caixa mostra "veio do cache"

EX 5 UMA CHAMADA SÓ

Tarefas em dias de chuva

Objetivo: buscar tudo de uma vez, em vez de uma chamada por tarefa.

- `findByConcluidaFalseAndDataPrazoBetweenOrderByDataPrazoAsc()`: as pendentes da janela.
- `PrevisaoService.buscarJanela()`: UMA chamada para os 16 dias, num mapa dia → previsão.
- `TarefaService.listarEmDiasDeChuva()`: chance de chuva de 60% ou mais no prazo. GET /tarefas/chuvosas.
- **5.1** 50 tarefas: quantas chamadas cada jeito faz? O que mais piora?
- **5.2** O GET /tarefas/chuvosas aproveita o cache do Exercício 4?

Pronto quando...

- Só pendentes, com a previsão do dia do prazo
- Chance de chuva de 60% ou mais
- O botão ☔ aparece no cadastro

Boas práticas de integração

Prática	Por quê
Isolar o cliente num pacote (client/)	Se o outro sistema mudar, só um lugar muda
O contrato do outro em records próprios	O formato dele não vaza para o nosso contrato
Ler só os campos que usa	Um campo novo do outro lado não quebra nada aqui
Validar antes de chamar	Pedido inválido não gasta uma requisição pela internet
Sempre configurar timeouts	Um serviço lento não prende a nossa API junto
Traduzir as exceções para as nossas	O resto do código não depende do RestClient
502 e 504, e não 500	Quem chama sabe que o problema está atrás da nossa API
Desconfiar do dado de fora	Código desconhecido, campo null, dia faltando: trate antes de usar

RECAPITULANDO

O que levamos para casa

- Integrar é a nossa API ser **cliente** de outra: no meio de uma requisição, ela faz outra.
- O contrato do outro é dado: fica isolado no client/, e o mapper traduz para o nosso.
- RestClient: get(), uri(), retrieve(), body(). Sem dependência nova, com timeouts no application.properties.
- As regras vêm antes da chamada. Falha do outro lado não é 400 nem 500: é **502** (falhou) ou **504** (demorou).
- Timeout quer dizer **"não sei"**: o pedido pode ter dado certo lá.

Dever de casa

Os 5 exercícios do EXERCICIOS.md (10 subexercícios), com todos os itens da contrato.html em ✅.

Entrega: código no GitHub + respostas no corpo do e-mail, assunto "Integração - <seu nome>".

Próxima aula: **Observabilidade e Testes**

MATERIAL DE APOIO

Para ir além

- RestClient, na documentação do Spring: docs.spring.io/spring-framework/reference/integration/rest-clients.html
- Clientes HTTP e timeouts no Spring Boot: docs.spring.io/spring-boot/reference/io/rest-client.html
- Open-Meteo, a documentação da API: open-meteo.com/en/docs
- Geolocalização no navegador: developer.mozilla.org (Geolocation API)
- Status HTTP 502 e 504: developer.mozilla.org (Web/HTTP/Status)
- Semântica do HTTP, status e idempotência (RFC 9110): rfc-editor.org/rfc/rfc9110
- Os exemplos desta aula: pasta Aula 12 do repositório github.com/placidodaniel/backend-aulas-satc

Experimente no navegador

api.open-meteo.com/v1/forecast?latitude=-28.68&longitude=-49.37&daily=weather_code&timezone=auto

...e troque daily=weather_code por current=temperature_2m